CS527 - Computer Systems Lab: OpenMP Parallel Programming

Name: Fardeen Khan Nabi Khan

Entry Number: 2026CSM1013

GitHub Repository Link: https://github.com/FARDeen9307/Computer-System-Parallel-Programming

1. Introduction & Overview

This report presents the implementation details, execution outputs, and performance benchmarking results for the OpenMP Parallel Programming lab tasks. The implementation includes vector addition, array reduction sums, min/max extreme tracking with indices, matrix addition (comparing outer-loop vs. collapse(2)), and naive matrix-matrix multiplication in C++.

Timing Rule Compliance

In strict adherence to high-performance computing timing guidelines:

- The wall-clock execution timer begins immediately before invoking the target parallel or serial function (omp_get_wtime()) and stops immediately after its completion.
- Dynamic memory allocation (new/delete[]), matrix initialization, verification checks, and terminal printing are completely excluded from the measured execution time.
- All reported runtime values represent stable, verified executions compiled with g++ -fopenmp -O2 -Wall.

2. Directory Structure & Execution Setup

2.1 Project Layout

Plaintext

```
Computer-System-Parallel-Programming/
│
├── lab_1_tasks/
│   ├── task1.cpp      # Vector Addition (Serial vs. Parallel)
│   ├── task2.cpp      # Array Sum Reduction & Naive Data Race
│   ├── task3.cpp      # Min/Max Value and Index Tracking
│   ├── task4.cpp      # Matrix Addition (Outer Loop vs. Collapse)
│   └── task5.cpp      # Matrix-Matrix Multiplication (Naive GEMM)
│
├── README.md          # Comprehensive Lab Execution Guide
└── .gitignore         # Excludes compiled binaries (.exe)
```

2.2 Execution Setup & Commands

Thread counts were managed dynamically prior to running the respective compiled binaries via PowerShell:

```
# Compilation Example
g++ -fopenmp -O2 task1.cpp -o task1


# Setting Thread Count & Execution
$env:OMP_NUM_THREADS = "8"
./task1
```

Task 1: Vector Addition

1. Objective & Problem Statement

The goal of this task is to get familiar with the basic OpenMP #pragma omp parallel for construct by implementing a serial and parallel vector addition of two arrays, $a[N]$ and $b[N]$, computing $c[i] = a[i] + b[i]$ for $N \geq 10{,}000{,}000$.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed using $env:OMP_NUM_THREADS.
- **Serial Version:** A standard sequential loop traversing indices from 0 to $N - 1$.
- **Parallel Version:** Parallelized using #pragma omp parallel for to distribute iterations across active threads.
- **Verification:** Element-wise comparison between serial and parallel result arrays (c_serial and c_parallel). Since floating-point addition for these initialized values is exact, exact identity is verified without tolerance margins, outputting SUCCESS.

- **3. Timing and Speedup Table**
- *(Measured across array sizes $N = 10^5, 10^6, 10^7, 10^8$ and thread counts $P = 1,2,4,8$)*

Array Size (N)	Threads (P)	Serial Time (Tserial)	Parallel Time (Tparallel)	Speedup (Tserial /Tparallel)	Verification
10^5	1	0.000s	0.000s	N/A (0s)	SUCCESS
	2	0.000s	0.000s	N/A (0s)	SUCCESS
	4	0.000s	0.001s	0.00	SUCCESS

	8	0.000s	0.001s	0.00	SUCCESS
10^6	1	0.005s	0.004s	1.25	SUCCESS
	2	0.005s	0.002s	**2.50**	SUCCESS
	4	0.003s	0.003s	1.00	SUCCESS
	8	0.003s	0.001s	**3.00**	SUCCESS
10^7	1	0.021s	0.023s	0.91	SUCCESS
	2	0.021s	0.013s	1.62	SUCCESS
	4	0.018s	0.010s	**1.80**	SUCCESS
	8	0.021s	0.012s	1.75	SUCCESS
10^8	1	0.315s	0.331s	0.95	SUCCESS
	2	0.265s	0.164s	1.62	SUCCESS
	4	0.259s	0.138s	1.88	SUCCESS
	8	0.256s	0.123s	**2.08**	SUCCESS

4. Questions to Address in Report

- **Question 1: At what array size does parallelization actually start paying off? Why is it not beneficial for small N?**
    - **Answer:** Parallelization typically starts showing a positive performance payoff when the array size reaches around $N = 10^6$ or higher. For smaller datasets, such as $N = 10^5$, parallel execution is generally slower or provides negligible benefit because the total execution time is already close to zero. At this small scale, the runtime overhead involved in spawning threads, managing task scheduling across processor cores, and handling thread synchronization vastly outweighs the actual time spent doing the math. Consequently, the system spends more time organizing the parallel tasks than performing the vector addition itself, making serial execution more efficient for small arrays.
- **Question 2: Does speedup keep increasing linearly with thread count? Why or why not?**
    - **Answer:** No, the speedup does not scale linearly with an increasing number of threads—for instance, using 8 threads does not yield an 8x performance improvement. This happens because vector addition is fundamentally a memory-bound operation rather than a compute-bound one, meaning its performance is restricted by how fast data can move between the system RAM and the CPU cache. As multiple threads execute simultaneously and attempt to read and write large arrays at the same time, they saturate the memory bus bandwidth. This memory bottleneck causes threads to wait for data access, creating a plateau in

performance where adding more threads no longer helps, aligning with the constraints described by Amdahl's Law.

Task 2: Sum of Array Elements

1. Objective & Problem Statement

The goal of this task is to learn how to use the OpenMP reduction clause and understand why a naive parallel summation approach is unsafe due to data races. The problem requires computing the total sum of all elements in an array $a[N]$ of size $N = 10^7$ and 10^8.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed using $env:OMP_NUM_THREADS.
- **Serial Version:** A sequential loop accumulating elements into a local sum variable.
- **Naive Parallel Version (Broken):** A parallelized loop using #pragma omp parallel for where every thread directly updates a shared sum += a[i] without synchronization, intentionally left broken to demonstrate race conditions.
- **Reduction Version:** A safe parallel version implemented using #pragma omp parallel for reduction(+:sum) to correctly accumulate results.
- **Verification:** Element-wise comparison ensuring the reduction-based sum matches the serial sum result across every run, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across array sizes $N = 10^7, 10^8$ and thread counts $P = 1,2,4,8$)

Array Size (N)	Threads (P)	Serial Time (Tserial)	Naive Time (Tnaive)	Reduction Time (Treduction)	Speedup (Tserial /Treduction)	Verification
10^7	1	0.008s	0.006s	0.007s	1.14	SUCCESS
	2	0.006s	0.005s	0.004s	**1.50**	SUCCESS
	4	0.006s	0.002s	0.003s	**2.00**	SUCCESS
	8	0.007s	0.003s	0.002s	**3.50**	SUCCESS
10^8	1	0.049s	0.050s	0.047s	1.04	SUCCESS
	2	0.039s	0.024s	0.026s	**1.50**	SUCCESS
	4	0.036s	0.025s	0.014s	**2.57**	SUCCESS
	8	0.042s	0.031s	0.013s	**3.23**	SUCCESS

4. Questions to Address in Report

- **Question 1: Why does the naive version (step 1) give wrong/inconsistent answers? Explain in terms of a data race on the shared variable sum.**
    - **Answer:** The naive parallel version produces incorrect and inconsistent results (as seen in the output where sums like 5,000,000 or 25,000,000 appear instead of the actual total) because of a data race on the shared sum variable. In C++, an operation like sum += a[i] is not atomic; it requires loading the current value of sum from memory into a register, adding a[i], and writing the result back. When multiple threads try to execute this read-modify-write sequence simultaneously on the exact same shared memory location without synchronization, updates made by one thread get overwritten or lost by another, leading to corrupt accumulation totals.
- **Question 2: Would using #pragma omp critical around sum += a[i] fix correctness? What would you expect to happen to performance if you did that instead of using reduction?**
    - **Answer:** Yes, wrapping the update inside #pragma omp critical would completely fix the correctness issue and ensure accurate results because it forces threads to enter the update section one at a time, preventing data races. However, the performance would degrade dramatically—often becoming much slower than the serial version. This happens because the critical section introduces heavy lock contention, forcing the parallel threads to continuously serialize at the lock entry point for every single array element. In contrast, the reduction clause avoids this by letting each thread accumulate its own private local sub-sum in parallel registers or cache, combining them only once at the very end of execution.

Task 3: Maximum and Minimum of an Array

1. Objective & Problem Statement

The goal of this task is to extend the reduction concept to max and min operations, and to practice finding not just the extreme value itself, but also the index at which that value occurs within an array $a[N]$ of size $N = 10^7$ and 10^8.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed using $env:OMP_NUM_THREADS.
- **Serial Version:** A sequential baseline loop scanning the array to track both maximum/minimum values and their corresponding indices.

- **Reduction Version:** Implemented using #pragma omp parallel for reduction(max:max_val) reduction(min:min_val) to efficiently find extreme values in parallel.
- **Critical Section Version:** An approach using #pragma omp critical to update a shared (best_val, best_idx) pair inside the loop to track both the value and its index.
- **Verification:** Checking that the parallel outputs match the serial baseline results across every test run, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across array sizes $N = 10^7, 10^8$ and thread counts $P = 1,2,4,8$)

Array Size (N)	Threads (P)	Serial Time (Tserial)	Reduction Time (Treduction)	Critical Time (Tcritical)	Speedup (Tserial /Treduction)	Verification
10^7	1	0.011s	0.016s	0.322s	0.69	SUCCESS
	2	0.012s	0.008s	0.650s	**1.50**	SUCCESS
	4	0.014s	0.006s	0.758s	**2.33**	SUCCESS
	8	0.017s	0.007s	1.099s	**2.43**	SUCCESS
10^8	1	0.114s	0.194s	3.363s	0.59	SUCCESS
	2	0.106s	0.092s	6.191s	**1.15**	SUCCESS
	4	0.094s	0.064s	7.477s	**1.47**	SUCCESS
	8	0.112s	0.052s	10.834s	**2.15**	SUCCESS

4. Questions to Address in Report

- **Question 1: Why can't you directly write reduction(max: best_idx) to get the index of the maximum?**
    - **Answer:** OpenMP built-in reduction clauses are strictly designed to combine scalar values using associative and commutative arithmetic operators (like +, max, or min). When tracking an index alongside a value, you are dealing with a dependent pair (value and index). If you apply max reduction on the value, OpenMP updates the maximum value correctly, but it has no built-in mechanism to tie or bind the corresponding array index update to that specific event without manual logic. A standalone index reduction would just find the maximum index value rather than the index belonging to the maximum array element.
- **Question 2: Compare the performance of the critical-section approach vs. the manual per-thread reduction approach for finding (max_val, max_idx). Which is faster, and why?**
    - **Answer:** The manual or reduction-based approach is vastly faster than the critical-section approach, as clearly reflected in the timing table where the critical method takes seconds

while reduction takes milliseconds. The critical section approach forces threads to constantly contend for a single global lock inside the inner loop for almost every iteration. This creates heavy lock contention and serializes the execution thread operations, causing performance to actually degrade severely as more threads are added. In contrast, the reduction or local chunk tracking allows each thread to work independently in its private cache without locks, merging results only at the very end.

Task 4: Matrix Addition

1. Objective & Problem Statement

The goal of this task is to move from 1D arrays to 2D matrices, and to compare the performance impact of parallelizing only the outer loop versus using the collapse(2) construct on matrix addition for $C[i][j] = A[i][j] + B[i][j]$.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed using $env:OMP_NUM_THREADS.
- **Serial Version:** Implemented using standard nested loops over a flattened 1D array representation of $N \times N$ matrices.
- **Outer-Loop Parallel Version:** Parallelized using #pragma omp parallel for exclusively on the outermost i loop.
- **Collapsed Parallel Version:** Implemented using #pragma omp parallel for collapse(2) across both i and j loops.
- **Verification:** Element-wise comparison confirming that both parallel implementations match the serial matrix result identically, outputting SUCCESS.

3. Timing and Speedup Table

Size	Threads	Tserial	Touter	Tcollapse	Speedup	Status
8	1	0.000s	0.000s	0.000s	N/A	SUCCESS
8	2	0.000s	0.001s	0.000s	0.00	SUCCESS
8	4	0.000s	0.000s	0.000s	N/A	SUCCESS

8	8	0.000s	0.001s	0.001s	0.00	SUCCESS
4096	1	0.039s	0.046s	0.054s	0.85	SUCCESS
4096	2	0.034s	0.024s	0.026s	1.42	SUCCESS
4096	4	0.040s	0.017s	0.020s	2.35	SUCCESS
4096	8	0.035s	0.025s	0.016s	1.40	SUCCESS

(Measured across matrix sizes $N = 8$ and $N = 4096$ and thread counts $P = 1,2,4,8$)

4. Questions to Address in Report

- **Question 1: For the small-N case, does collapse(2) help compared to parallelizing only the outer loop? Explain why, in terms of how many independent iterations are available to distribute across threads in each version.**
    - **Answer:** For the small-N case (such as $N = 8$), neither approach provides a measurable performance benefit because the total execution time is essentially zero and completely dominated by thread overhead. In terms of iterations, parallelizing only the outer loop provides just $N = 8$independent iterations, which cannot be efficiently distributed across multiple cores (like 4 or 8 threads). Using collapse(2) expands the iteration space to $N \times N = 64$iterations. However, because the workload is so tiny, the runtime overhead of scheduling even 64 iterations across threads still outweighs any potential concurrency advantage, keeping times negligible.
- **Question 2: For the large-N case, does collapse(2) still make a measurable difference, or is parallelizing the outer loop alone already enough to keep all threads busy?**
    - **Answer:** For the large-N case ($N = 4096$), parallelizing the outer loop alone provides 4096independent iterations, which is already more than enough to fully occupy standard thread counts (1, 2, 4, or 8 threads). Consequently, outer-loop parallelization alone achieves good speedups (up to 2.35x). However, collapse(2) combines the loops into a massive iteration space of $4096 \times 4096 \approx 1.67 \times 10^7$iterations, which can sometimes provide slightly better load balancing or finer-grained scheduling across threads at higher thread counts, though both approaches perform similarly as the problem size scales.

Task 5: Matrix-Matrix Multiplication (Naive)

1. Objective & Problem Statement

The goal of this task is to parallelize the classic triple-nested-loop matrix multiplication algorithm ($C = A \times B$) for $N \times N$ matrices and study how the choice of parallelized loop configuration and loop collapsing affects execution performance and scalability.

2. Implementation Details:

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed using $env:OMP_NUM_THREADS.
- **Serial Version:** A standard triple-nested loop computing matrix multiplication with zero-initialization of matrix C and optimized local sum variable accumulation.
- **Outer-Loop Parallel Version:** Parallelized using #pragma omp parallel for exclusively on the outermost i loop.
- **Collapsed Parallel Version:** Implemented using #pragma omp parallel for collapse(2) across the i and j loops, utilizing local accumulation.
- **Verification:** Tolerance-based floating-point verification checking that parallel outputs match the serial baseline across all matrix elements, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across matrix sizes $N = 256,512,1024$ and thread counts $P = 1,2,4,8$)

Size (N)	Threads (P)	Tserial	Touter	Tcollapse	Outer Speedup	Collapse Speedup	Status
256	1	0.024s	0.014s	0.014s	1.71	1.71	SUCCESS
	2	0.020s	0.008s	0.007s	2.50	2.86	SUCCESS
	4	0.020s	0.004s	0.009s	5.00	2.22	SUCCESS
	8	0.021s	0.008s	0.006s	2.63	3.50	SUCCESS
512	1	0.450s	0.401s	0.434s	1.12	1.04	SUCCESS
	2	0.397s	0.187s	0.182s	2.12	2.18	SUCCESS

	4	0.401s	0.105s	0.099s	3.82	4.05	SUCCESS
	8	0.383s	0.058s	0.054s	6.60	7.09	SUCCESS
1024	1	6.121s	4.111s	4.187s	1.49	1.46	SUCCESS
	2	5.926s	2.306s	2.051s	2.57	2.89	SUCCESS
	4	6.157s	1.259s	0.983s	4.89	6.26	SUCCESS
	8	5.299s	0.699s	0.699s	7.58	7.58	SUCCESS

4. Questions to Address in Report

- **Question 1: Why is it safe to parallelize the i loop directly (no race condition), even though the innermost k loop is doing a summation into C[i][j]?**
    - **Answer:** It is completely safe to parallelize the outer i loop directly because each iteration of the i loop is entirely independent. Every thread handles a unique, separate row of the result matrix C (C[i][j]), meaning no two threads ever write to or modify the same memory location. Although the inner k loop performs a summation, that accumulation is isolated to the specific row and column being computed by that particular thread's assigned i index, completely eliminating any cross-thread data races. Furthermore, using a local sum variable within the loop ensures that all intermediate accumulation happens inside fast CPU registers rather than triggering constant conflicting memory updates.
- **Question 2: What happens to correctness if you try to parallelize the k loop directly with a plain parallel for (no reduction)? Explain the race condition that would occur on C[i][j].**
    - **Answer:** If you attempt to parallelize the inner k loop directly using a plain parallel for without a reduction clause, the program will produce severely incorrect and corrupted results due to a severe data race. In this scenario, multiple threads would simultaneously try to access, read, compute, and write back to the exact same shared memory locations of C[i][j] across different iterations of k. Since a write operation like C[i][j] += A[i][k] * B[k][j] is not atomic, threads will constantly overwrite or lose each other's intermediate accumulation steps, completely destroying the integrity of the final matrix product.
- **Question 3: Compare parallel for on i alone vs. collapse(2) on (i, j). For which values of N (relative to your thread count) would you expect collapse(2) to matter most?**
    - **Answer:** Comparing outer-loop parallelization against collapse(2) on (i, j), the collapsed version matters most when the matrix size N is relatively small compared to the available

thread count (or when N is moderately small, such as $N = 256$). When N is small, parallelizing only the outer loop provides very few iterations (N total rows), which is insufficient to adequately feed a high number of threads, leaving some processor cores idle. Using collapse(2) combines the outer and middle loops into a vastly expanded iteration space ($N \times N$), providing a large pool of independent tasks that allows OpenMP to distribute the workload much more evenly across all active threads and improve efficiency. For very large N (such as $N = 1024$), both approaches perform similarly because the outer loop alone already provides thousands of iterations, which is more than enough to keep all threads fully occupied.